

BCSE – 3rd year – 1st Semester – 2026-27
Assignment – II [Q1-Q3: CO2, Q4-Q5: CO3]
Operating Systems Laboratory

1. Consider a pair of processes X and Y, which share the variables x and y. Process X writes x and reads y while process Y writes y and reads x. Y must not read x until X has written to it and X must not read y until Y has written to it. Implement this using pipe/s.
2. Create child processes: X and Y.
 - a. Each child process performs N iterations ($N \geq 10$, being an input to be provided by user). The child processes display their name/id and the current iteration number, and sleep for some random amount of time. Adjust the sleeping duration of the processes to have a different output (i.e. another interleaving of processes' traces).
 - b. Modify the program so that process Y is not allowed to start iteration i before process X has terminated its own iteration $i-1$.
3. Implement the following applications using different IPC mechanisms. Your choice is restricted to Pipe/FIFO/Message Queue (use different mechanisms for each program)
 - a. Broadcasting weather information (one broadcasting process and more than one listeners)
 - b. Telephonic conversation (between a caller and a receiver)
 - c. One broadcasting process and more than one listeners where listeners may also say something to the broadcaster
4. Write down scheduler for jobs whose execution profiles are in a file that is to be read as input to the scheduler.

Format of the job profile:

<Job-id priority arrival-time CPU-burst(1) I/O-burst(1) CPU-burst(2)..... -1>

(Each information is separated by blank space and each job profile ends with -1. Lesser priority number denotes higher priority process with priority number 1 being the process with highest priority.)

Example: 2 3 0 32 64 16 32 -1 1 2 4 64 8 -1 etc.

Testing:

- a. There must be 8 job profiles in the file.
 - b. Test with each of the following scheduling algorithms:: FCFS, Non-preemptive Priority and Round Robin (time slice:16).
 - c. Compare the average waiting time, turnaround time of each process for the different scheduling algorithms.
5. Write a program that will find out whether a system is in safe state or not with following specifications:

Command line input:: name of a file containing, for example::

<#no of resources> 4 <#no of instances of each resource> 2 4 5 3

<#no of processes> 3 <#no of instances of each resource that each process needs in its lifetime separated by “,”> 1 1 1 1, 2 3 1 2, 2 2 1 3

Present situation: Sequence <pid r0 r1 r2 r3> to be repeated for each process with a “,” separator between pids. Resource requests by processes also follow the same sequence.

- a. Generate a present situation of resources allocated to each process and possible request sequences of each process.
- b. Each such sequence must satisfy the maximum requirements of the process.

Your program should declare the result:

(1) should the request be granted?

(2) if your answer is yes, print the safe sequence. Go back to accept another request till all processes finish with all their needs.

Date for submission [and viva] latest by: September 21st/23rd/24th 2026 (respective groups)